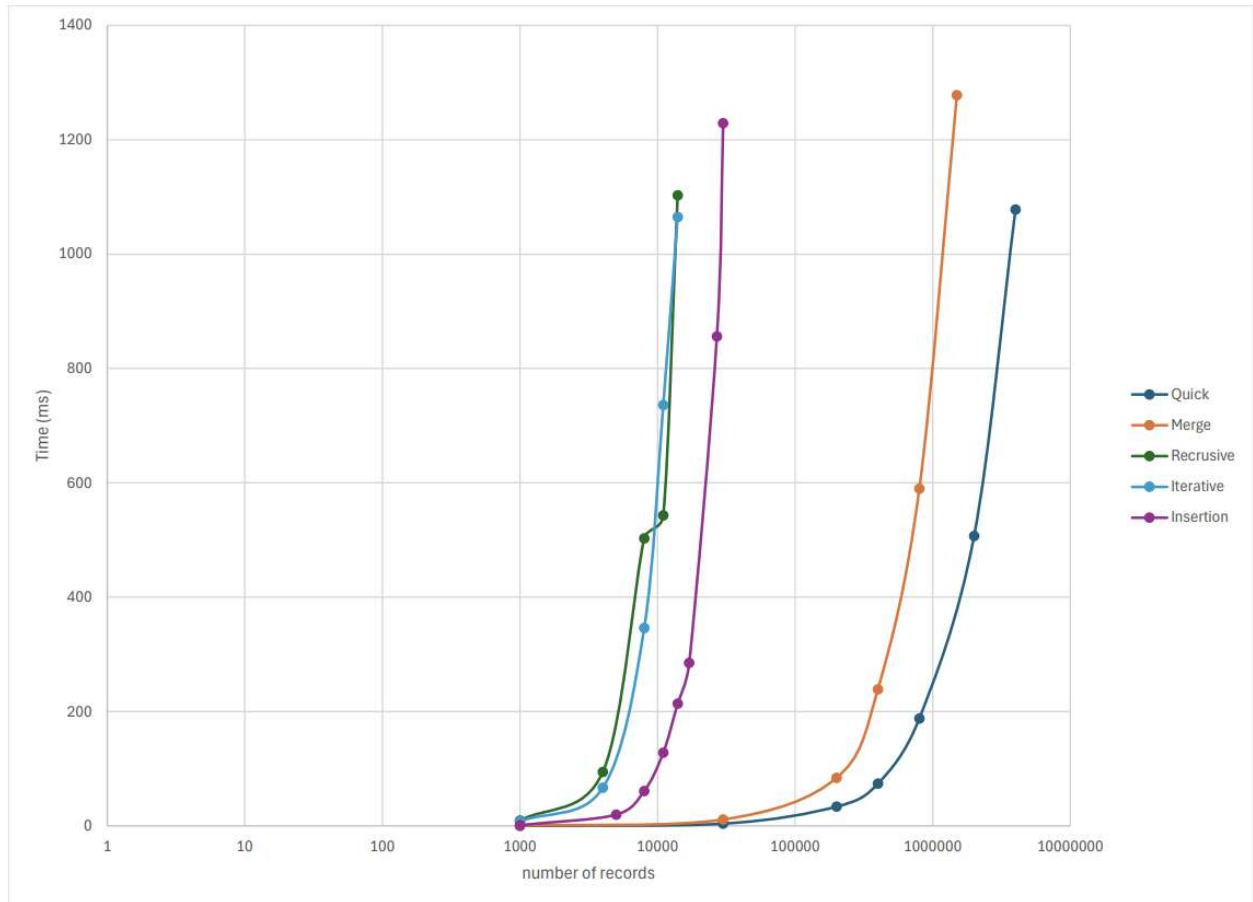




1)

For random lists, the slowest was recursive and iterative selection sort. This was due to the need to approximately compare each element in the list against the majority of the list. For this reason, the algorithms have complexity $O(n^2)$. The next slowest was insertion sort which on average ran over each element of the list a fraction of n times resulting in average complexity $O(n^2)$. Finally, merge and quicksort. The data shown above shows that merge and quicksort have similar curves. While I won't restate the derivation of their complexities here, in ideal cases both of these algorithms have complexity $O(n \log n)$ which would explain why they were so much faster than the 3 other algorithms written.

2)

If the list is already sorted, algorithms which can efficiently confirm the list is sorted in the correct manner will often perform much faster than their average complexities. For example, insertion sort which usually runs in $O(n^2)$ is reduced in complexity to $O(n)$ as the only operations needing to be performed is to take the front of the list and insert it into the back of the new list, a constant amount of work per element.